

CS223 Computer Architecture & Organization

Cache Memory – Block Replacement Techniques



John Jose

Associate Professor

Department of Computer Science & Engineering

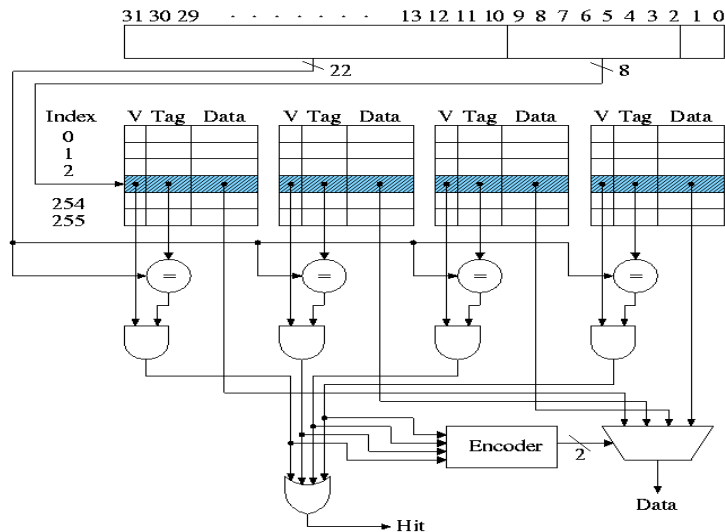
Indian Institute of Technology Guwahati

Four cache memory design choices

- ❖ Where can a block be placed in the cache?
 - **Block Placement**
- ❖ How is a block found if it is in the upper level?
 - **Block Identification**
- ❖ Which block should be replaced on a miss?
 - **Block Replacement**
- ❖ What happens on a write?
 - **Write Strategy**

Block Replacement

- ❖ Cache has finite size. What do we do when it is full?
- ❖ Direct Mapped is Easy
- ❖ Which block to be replaced for a set associative cache?



Block Replacement Algorithms

- ❖ Random
- ❖ First In First Out (FIFO)
- ❖ Last In First Out (LIFO)
- ❖ Least Recently Used (LRU)
- ❖ Pseudo-LRU (PLRU)
- ❖ Least Frequently Used (LFU)
- ❖ Optimal

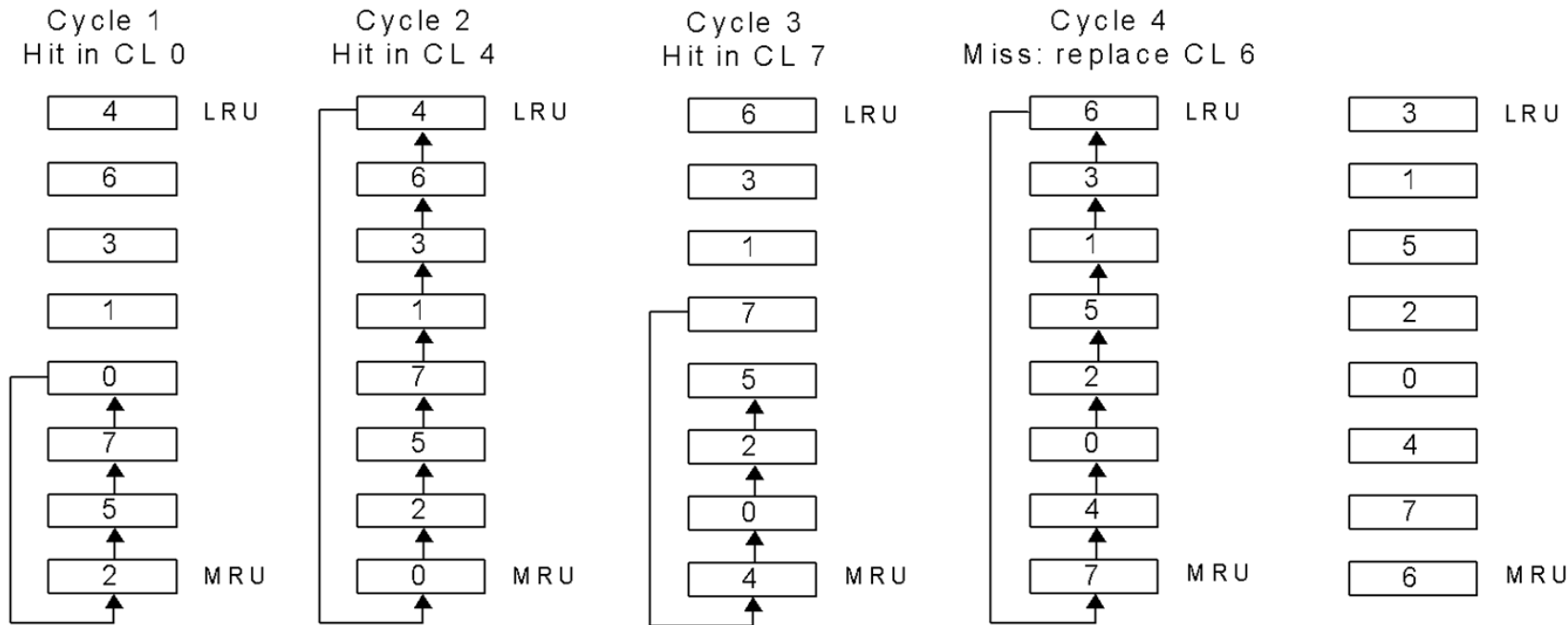
Random & FIFO Replacement Policy

- ❖ **Random policy** needs a pseudo-random number generator
- ❖ Makes no attempt to take advantage of any temporal or spatial localities
- ❖ **First-in, First-out(FIFO) policy** evict the block that has been in the cache the longest
- ❖ It requires a queue Q to store references
- ❖ Blocks are enqueued in Q, dequeue operation on Q to determine which block to evict.

Least-Recently Used

- ❖ For associativity = 2
 - ❖ Single bit per line indicates LRU/MRU
 - ❖ Set/clear on each access
- ❖ For $a > 2$, LRU is difficult/expensive
 - ❖ Record Timestamps? How many bits?
 - ❖ Must find min timestamp on each eviction
 - ❖ Sorted list? Re-sort on every access?
 - ❖ Shift register implementation

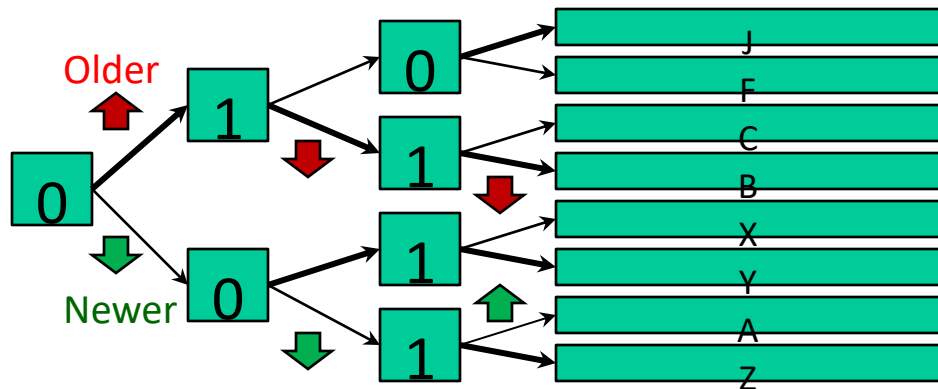
LRU Implementation



Optimal Replacement Policy

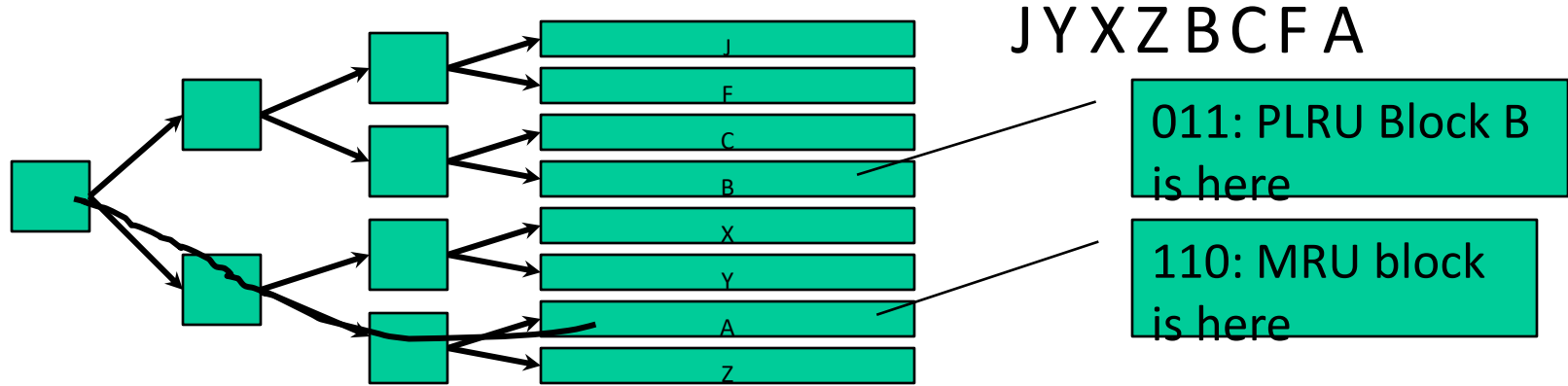
- ❖ Evict block with longest reuse distance
 - ❖ i.e. next reference to block is farthest in future
 - ❖ Requires knowledge of the future!
- ❖ Can't build it, but can model it with trace
- ❖ Useful, since it reveals opportunity
- ❖ Optimal better than LRU
 - ❖ (X,A,B,C,D,X): LRU 4-way SA cache, 2nd X will miss

Practical Pseudo-LRU

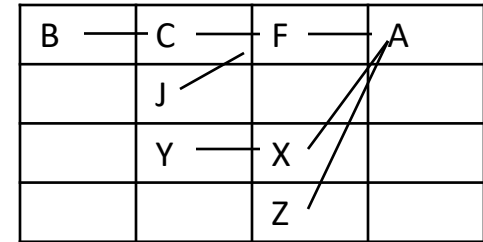
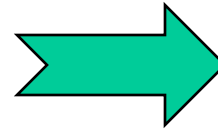
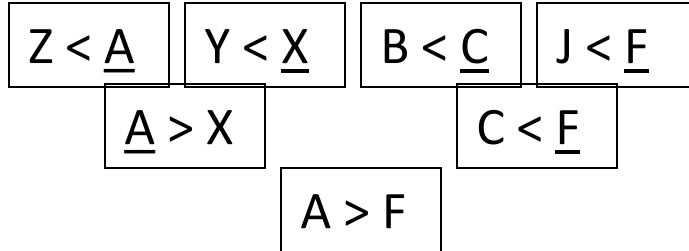


- ❖ Rather than true LRU, use binary tree
- ❖ Each node records which half is older/newer
- ❖ Update nodes on each reference
- ❖ Follow older pointers to find LRU victim

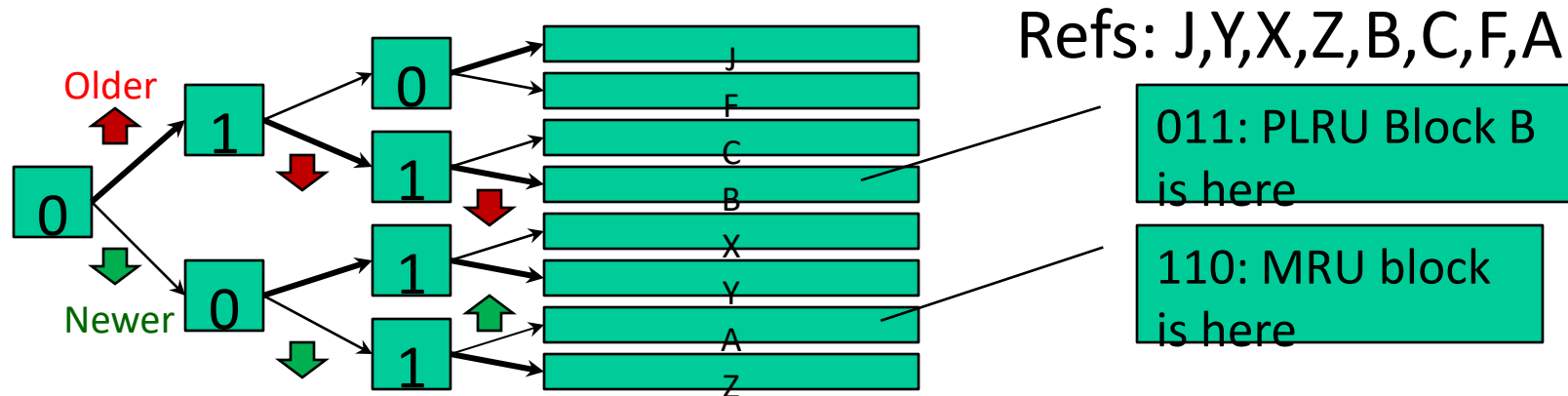
Practical Pseudo-LRU



Partial Order Encoded in Tree:



Practical Pseudo-LRU



- ❖ Binary tree encodes PLRU partial order
- ❖ At each level **point** to LRU half of subtree
- ❖ Each access: flip nodes along path to block
- ❖ Eviction: follow **LRU** path
- ❖ Overhead: $(a-1)/a$ bits per block

Least Frequently Used

- ❖ Counter per block, incremented on reference
- ❖ Evictions choose lowest count
- ❖ Logic not trivial (a^2 comparison/sort)
- ❖ Storage overhead
 - ❖ 1 bit per block: same as NRU
 - ❖ How many bits are helpful?

Cache Block Replacement Exercises

- ❖ How many conflict misses are encountered when FIFO cache block replacement technique is used with a 4-way set associative cache for the following block access pattern all indexing to the same set. Assume initially the cache is empty.

P, Q, R, S, T, P, Q, S, R, T, Q, P

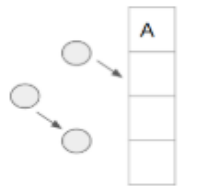
Block	P	Q	R	S	T	P	Q	S	R	T	Q	P
Way-1	P	P	P	P	T	T	T	T	T	T	T	T
Way-2	-	Q	Q	Q	Q	P	P	P	P	P	P	P
Way-3	-	-	R	R	R	R	Q	Q	Q	Q	Q	Q
Way-4	-	-	-	S	S	S	S	S	R	R	R	R

Cache Block Replacement Exercises

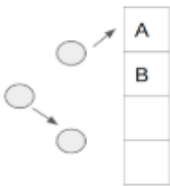
- ❖ The following 13 memory block requests **A, B, D, A, B, C, E, A, B, E, D, C & D** are mapped to set n of a 4-way set-associative cache memory that uses Practical Pseudo LRU block replacement technique. Assume that set n is initially empty. What will be the contents of set n (in the order way0, way1, way2 and way3) after servicing all the requests? [Assume that data is entered into an empty cache block from way-0, way-1, way-2 & way-3 order]

Cache Block Replacement Exercises

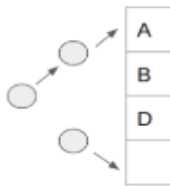
❖ A, B, D, A, B, C, E, A, B, E, D, C, D



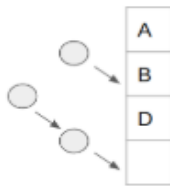
A(compulsory miss)



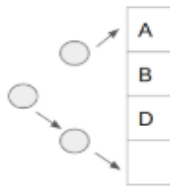
B(compulsory miss)



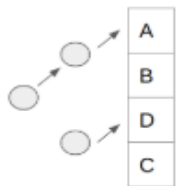
D(compulsory miss)



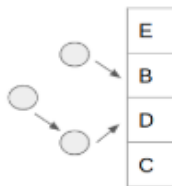
A(hit)



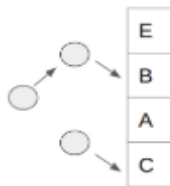
B(hit)



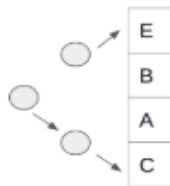
C(compulsory miss)



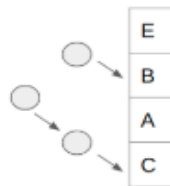
E(compulsory miss)



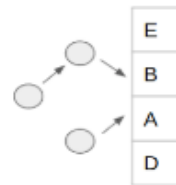
A(conflict miss)



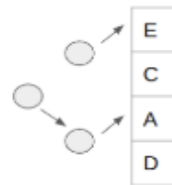
B(hit)



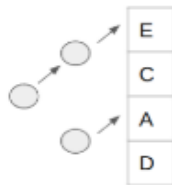
E(hit)



D(conflict miss)



C(conflict miss)



D(hit)

- Total access = 13
- 5 compulsory misses
- 3 conflict misses
- Hits= 5
- Final contents → ECAD
- from way-0 to way-n.



johnjose@iitg.ac.in
<http://www.iitg.ac.in/johnjose/>